

Nabeela Khan

Dr.Clark

Advanced C++

October 23rd, 2024

Lab 2: Profiler System Report

The profiler system designed was to measure the code performance by tracking the execution time of various sections within the code. It uses a modular structure and is divided into several key classes.

Profiler System Design:

Overview of the architecture:

There are 4 main classes that the profiler system manages. There is the Profiler class, the TimeRecordStart, TimeRecordStop, ProfilerStats, and ProfilerScopeObject. There is also a time.cpp file but this mainly just gets the current time in seconds using Chrono.

Profiler Class: This class is responsible for tracking the start and stop times, recording the execution times, and calculating and reporting the statistics.

The data structures in this class include the vector(startTimes) that stores the start times of each profiled section. There is another vector called elapsedTimes that records the elapsed time for each section once it is completed. The map which is called stats holds the detailed statistics (ProfilerStats) for each section, including the total execution time, count, and min/max times.

Below are the most important functions of the entire profiler system:

Profiler::EnterSection(const char* sectionName)

- This function is called at the start of every code section that needs to be profiled. It performs the following steps:
- A std::lock_guard is used to ensure exclusive access to shared data structures like startTimes and elapsedTimes.
- The function calls GetCurrentTimeSeconds() to capture the current time. This start time is stored in the startTimes vector, associated with the provided section name.
- The section name is pushed onto the activeSections stack to track which sections are currently running, enabling hierarchical profiling.

Profiler::ExitSection(const char* sectionName)

- This function is called when a section finishes execution. It performs the following actions:
- Like EnterSection(), a std::lock_guard ensures thread-safe access.

- It iterates through the startTimes vector (from the most recent entry backward) to find the matching start time for the section. Once found, the elapsed time is calculated as the difference between the current time (GetCurrentTimeSeconds()) and the recorded start time.
- The elapsed time is added to the elapsedTimes vector, along with the section name and other debugging information.
- The ReportSectionTime() function is called to update the profiling statistics for this section.
- If the section is not found in startTimes, a warning is printed to notify the developer of potential mismatches in start/stop profiling.

Profiler::ReportSectionTime()

- This function records the elapsed time for a specific section of code and updates the associated statistics in ProfilerStats. It is overloaded to handle both cases:
 - With only the elapsed time (without extra debugging information)
 - With full debugging information (file name, function name, and line number).
- This function updates or creates a new ProfilerStats entry for the section in the stats map, incrementing the execution count and adjusting the total time, min/max times, and average time.

Profiler::calculateStats()

- This function iterates over the recorded elapsedTimes and updates the ProfilerStats for each section. It ensures that the statistics are consistent and accurate:
- It calculates the average execution time by dividing the total time by the number of calls (count).
- It adjusts the minimum and maximum times based on the recorded elapsed times for the section.
- This method processes all profiling data after the profiling session is complete, minimizing overhead during profiling.

ProfilerStats: This class is responsible for storing profiling data for each individual section of code it tracks.

It tracks count(the count of how many times the section has been executed), total time(the total amount of time spent in the section across all executions), min/max time(the shortest/longest execution times for the section), and averageTime(the average execution time).

The ProfilerStats class also records metadata about the code section, such as the file name, function name, and line number, allowing developers to pinpoint the exact location of performance issues.

TimeRecordStart and **TimeRecordStop:** These classes are used to track the precise time at which a section of code starts and stops.

TimeRecordStart records the section name and the start time (in seconds) when the section begins.

TimeRecordStop tracks the section name, elapsed time (the difference between start and stop times), and additional debugging information (like file name, function name, and line number) when the section ends.

These objects are used to record timing information in the startTimes and elapsedTimes vectors, respectively.

ProfilerScopeObject: This class is a utility designed to automatically handle entering and exiting sections. It uses the RAII (Resource Acquisition Is Initialization) pattern:

When a ProfilerScopeObject is created, it immediately starts profiling a section.

When it goes out of scope (for instance, at the end of a function or block), it automatically exits the section, ensuring that the profiling ends correctly even if an exception occurs. This class is particularly useful for reducing manual code management and providing cleaner and safer profiling integration.

Performance considerations:

The main consideration when making the profiler system was to make sure that it was very lightweight. To do this, it was important to keep low overhead for time tracking. In order to implement this, we used high-resolution clocks with `std::chrono`. This ensured that time measurements were accurate for even short sections of code. The `startTimes` and `elapsedTimes` were stored in a `std::vector` which offers fast access and insertion times. Also, the constructor of Profiler already reserved memory for these vectors upfront which essentially improves the performance of the profiler.

To also have minimal overhead for profiling operations, lightweight macros were implemented. This approach made it easy to add profiling markers in the code without complex function calls and minimizing overhead. Another way we ensured that the profiler system was lightweight was that we made sure that we only calculateStats in the end and not every time we call the `ExitSectionMethod`. Having a `calculate stats` method makes sure that the operations of calculating stats do not interfere with the actual profiling process. In addition, the `std::mutex` ensures thread-safe access to shared data structures such as `startTimes`, `elapsedTimes`, and `activeSections`. This ensured that multiple threads could safely profile sections of code without causing data corruption or inconsistency. Lastly, in the destructor of the Profiler, resources such as `ProfilerStats` are cleaned up to avoid memory leaks. However, the system does not over-manage memory, ensuring minimal performance overhead in terms of memory management.

Guide to use Profiler API:

1. Initializing the Profiler

You have to first include the `profiler.hpp` class in your `main.cpp` file like this: `include "profiler.hpp"`

Before you can start profiling, you need to initialize the Profiler system. This should be done at the start of your `main()` function.

How to Initialize the Profiler

cpp

Copy code

```
int main() {
    // Initialize the global profiler instance
    Profiler* profiler = Profiler::GetInstance();

    // Check if initialization succeeded
    if (!profiler) {
        std::cerr << "Failed to initialize Profiler." << std::endl;
        return 1;
    }
}
```

```

// Your code here (start profiling sections, etc.)

// Cleanup the profiler instance at the end of the program
delete profiler;
return 0;
}

```

This ensures the Profiler is ready to be used throughout your application.

2. Profiling Sections of Code

The Profiler allows you to mark sections of code that you want to measure. You use two macros: one to start profiling and one to stop profiling.

Start Profiling a Section (PROFILER_ENTER):

To start profiling a section of code, use the PROFILER_ENTER() macro and give the section a name (as a string):

Stop Profiling a Section (PROFILER_EXIT):

To stop profiling the section, use the PROFILER_EXIT() macro, passing the same section name:

Example: Profiling a Function

cpp

Copy code

```

void MyFunction() {
    PROFILER_ENTER("MyFunction Execution");

    // Code you want to profile
    for (int i = 0; i < 1000; ++i) {
        // Some operations
    }

    PROFILER_EXIT("MyFunction Execution");
}

```

This example will measure how long it takes to execute the function MyFunction.

3. Calculating and Printing Profiling Statistics

Once you have profiled sections of your code, you can calculate and display the statistics.

How to Calculate and Print Stats

After all your profiling is done (for example, at the end of main()), call these methods to calculate the stats and print them:

```
// Calculate the profiling statistics  
profiler->calculateStats();
```

```
// Print the statistics to the console  
profiler->printStats();
```

This will print out the following for each profiled section:

Total time
Minimum time
Maximum time
Average time
Number of times the section was executed

4. Exporting Profiling Data (Optional)

If you want to save the profiling data to a file (e.g., CSV or JSON) for further analysis, you can use these methods:

Export to CSV
profiler->printStatsToCSV("profiler_stats.csv");

Export to JSON
profiler->printStatsToJSON("profiler_stats.json");

Hierarchical and Interleaving Profiling:

Hierarchical profiling: this allows you to measure sections of code that are nested inside each other. In the code, the PROFILER_ENTER() macro is used to start profiling for a specific section and the PROFILER_EXIT() macro ends it. If another PROFILER_ENTER() call is inside an already active section, the system treats it as a nested section.

The Profiler keeps track of the active sections using a stack called activeSections. This stack allows the system to manage nested entries, pushing each new section onto the stack and popping it off when it's complete.

How the Profiler Handles Hierarchy:

- The Profiler uses the PROFILER_ENTER and PROFILER_EXIT macros to start and stop tracking sections of code.
- When a new section starts inside an already active section, it is treated as a nested section.
- The Profiler maintains a stack (activeSections) to track active sections, ensuring that the most recently started section is always the one that gets stopped first.

- When a section ends, the Profiler calculates the elapsed time by looking up the start time from the reverse-ordered startTimes vector. This allows the system to close the innermost section first, followed by the outer sections.

Example (taken from my main.cpp file):

```
void Test1() {
    PROFILER_ENTER("Trig Speed Test");

    // Enter section for generating random angles
    PROFILER_ENTER("Random Angle Generation");
    // Code to generate random angles
    PROFILER_EXIT("Random Angle Generation");

    // Enter section for total cosine and sine computations
    PROFILER_ENTER("Total Cos and Sin Compute");
    for (int i = 0; i < 100000; i++) {
        PROFILER_ENTER("Cos and Sine Compute");
        // Code to compute cosine and sine
        PROFILER_EXIT("Cos and Sine Compute");
    }
    PROFILER_EXIT("Total Cos and Sin Compute");

    PROFILER_EXIT("Trig Speed Test");
}
```

1. Outer Section: PROFILER_ENTER("Trig Speed Test") starts the profiling for the entire trig speed test.
2. Nested Sections: The random angle generation and cosine/sine computations are nested within the overall trig test.
3. The Profiler tracks each section independently, allowing you to see both the total time for the outer section and the time spent in each inner task.

Interleaving Profiling: This occurs when sections of code alternate in execution, often overlapping in time. This happens in concurrent or parallel tasks where one section starts, another section runs in between, and then the first section resumes or ends.

How Profiler Handles Interleaving:

- The Profiler can handle sections that start and stop independently, even if they overlap in time.
- By iterating through the startTimes vector in reverse order when stopping a section, the Profiler ensures that the most recently started section is closed first, even if other sections remain active.
- This allows for accurate measurement of tasks that are interleaved or alternate in execution without interference from one another.

Example (taken from my main.cpp file):

```
void RunInterleavedTest() {  
    PROFILER_ENTER("Task A");  
  
    for (int i = 0; i < 5; ++i) {  
        PROFILER_ENTER("Task B"); // Start Task B  
        // Code for Task B  
        PROFILER_EXIT("Task B"); // End Task B  
    }  
  
    PROFILER_EXIT("Task A");  
}
```

In this example:

1. Task A is the outer task, and it stays active during the loop.
2. Task B starts and stops multiple times within the execution of Task A.
3. This is an example of interleaving because Task B is repeatedly started and stopped while Task A is still running.

Visualization Tool:

For my visualization tool, I chose to use streamlit and plotly for my core technologies.

I chose Streamlit because it allows developers to create interactive web applications with minimal code. It makes it ideal for quickly turning data into visual insights. Streamlit also seamlessly integrates data processing libraries like Pandas, facilitating efficient data handling and manipulation. It also supports real-time updates and interactive widgets, enhancing user engagement and exploration of data.

I chose Plotly because it has interactive graphs that excel in creating highly interactive and aesthetically pleasing graphs, enabling users to hover, zoom, and filter data dynamically. With support for a wide range of chart types, Plotly provides the flexibility needed to represent diverse data aspects effectively. Plotly also offers extensive customization options, allowing for tailored visualizations that align with specific reporting needs.

Visuals:

Graph 1: Elapsed Time(ms) by Section

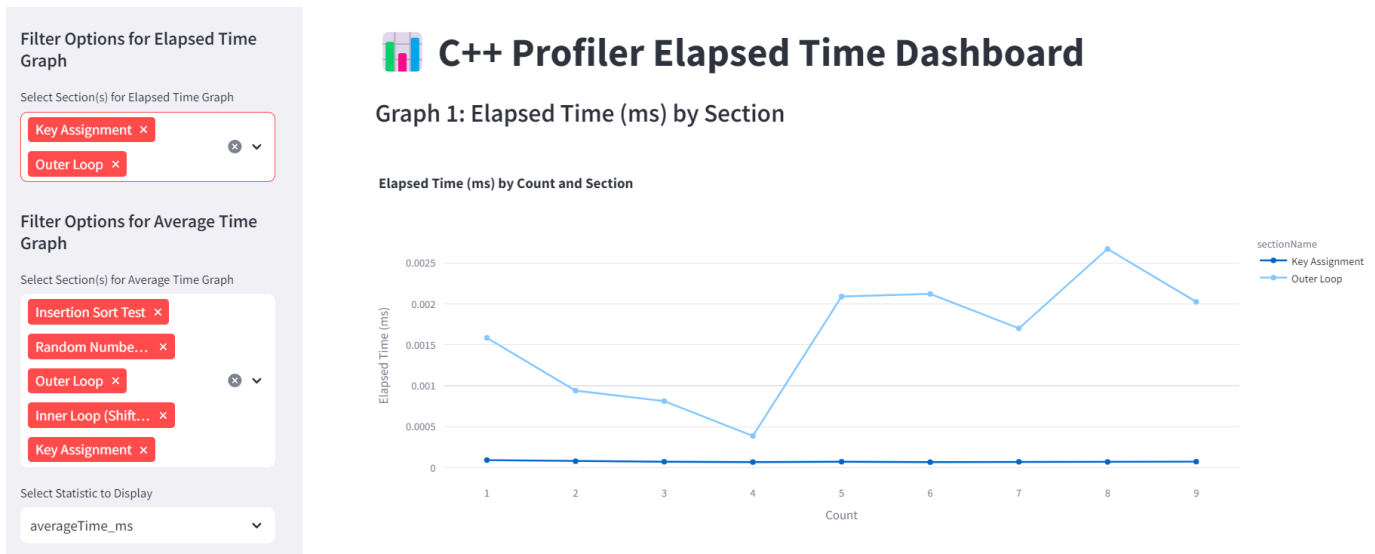
This line graph displays the elapsed time for each profiled section across multiple executions. It allows users to observe how the execution time of specific sections varies over different runs or iterations.

Features:

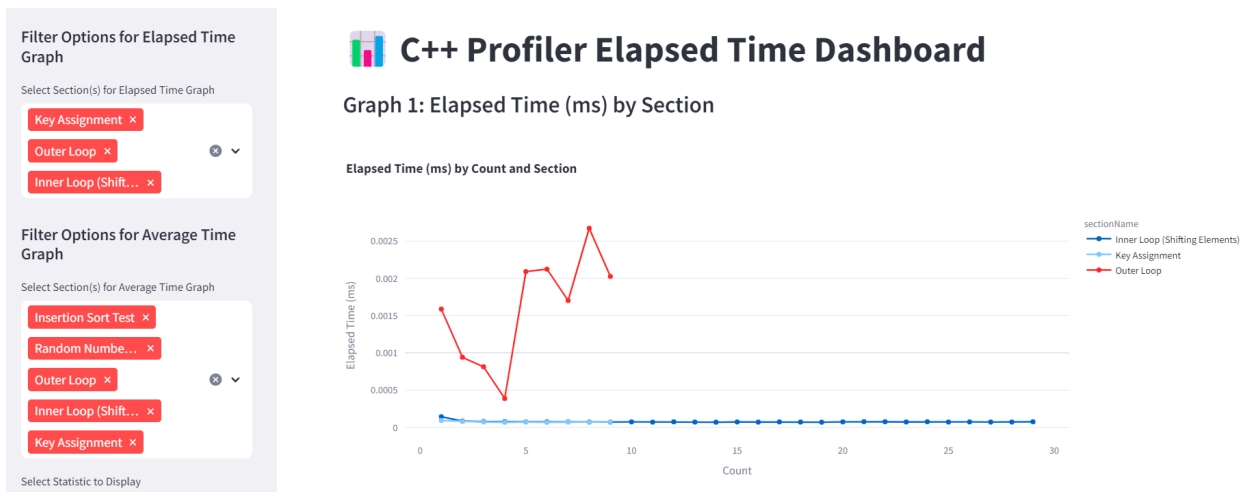
- Interactive filtering: users can select one or more sections to focus on, enabling targeted analysis of specific code areas
- Dynamic count axis: the x-axis represents the count, while the y-axis shows the elapsed time in milliseconds

- **Hover Tooltips:** Hovering over data points reveals detailed statistics, including the exact elapsed time, line number, function name, and more.

This is an example of choosing two section names and seeing the graph:



Adding the section inner loop shift:



Graph 2: Average/Min/Max Execution Time per Section

This bar chart illustrates the average/min/max execution time (in milliseconds) for each profiled section. You can select which specific statistic you want to see for the section which includes min, max, and average.

Features:

- **Section Selection:** Users can select specific sections to include in the graph, allowing for focused performance analysis.
- **Statistic Selection:** A dropdown menu enables users to choose which statistic to display (total time, minimum time, maximum time, or average time).
- **Sorting Options:** Users can sort the bars in ascending or descending order based on the selected statistic, facilitating easy identification of the most or least time-consuming sections.

- Color Coding: Bars are color-coded based on the magnitude of the selected statistic, enhancing visual differentiation.

Showing the average time for all the sections;



Changed it to min time and made the graph ascending:



Graph 2: Average Execution Time per Section

minTime_ms per Section (ms)

Section Name	minTime_ms
Key Assignment	0.001
Inner Loop (Shifting Elements)	0.001
Outer Loop	0.001
Random Number Generation	0.001
Insertion Sort Test	0.04

minTime_ms

0.04

0.03

0.02

0.01

0

All the statistics available to see:

Select Statistic to Display

minTime_ms

totalTime_ms

minTime_ms

maxTime_ms

averageTime_ms

User Guide:

1. Ensure that the Profiler has been executed and the CSV data files (detailedStats.csv and profiler_stats.csv) are available in the Data/ directory.
2. Run the Streamlit application by executing the following command in your terminal:
`streamlit run Code/combinedGraphs.py`
3. The dashboard will open in your default web browser, typically accessible at <http://localhost:8501/>.
4. Interacting with Graph 1: Elapsed Time by Section
 - a. Navigate to the sidebar on the left side of the dashboard.
 - b. Under "Filter Options for Elapsed Time Graph", you will find a multiselect widget labeled "Select Section(s) for Elapsed Time Graph".
 - c. Select one or more sections you wish to analyze. By default, all sections are selected.
 - d. Hover over the data points to get detailed information about elapsedTime, sectionName, line number, and function name.
5. Interacting with Graph 2: Average Execution Time Per Section:
 - a. In the sidebar, under "Filter Options for Average Time Graph", locate the multiselect widget labeled "Select Section(s) for Average Time Graph".
 - b. Choose the sections you want to include in the analysis. All sections are selected by default.
 - c. Use the dropdown menu labeled "Select Statistic to Display" to choose which metric to visualize: totalTime_ms: Total execution time, minTime_ms: Minimum execution time, maxTime_ms: Maximum execution time, averageTime_ms: Average execution time
 - d. You can also select the desired sort order (descending or ascending) using the drop down menu labeled "sort order"
6. After selecting all your preferences, Graph 2 should display a bar chart with the chosen statistic for each selected section.
7. The bars are also color-coded using the viridis color scale with color intensity reflecting the magnitude of the statistic

Overall, the visualization's main goal is to demonstrate a summary and analysis of the performance data. It allows users to delve deeper into specific data points and facilitate targeted performance analysis. It also helps with providing real-time insights.

Code Demonstration:

Original code:

The InsertionSortTest2() function implements the insertion sort algorithm, which sorts an array of random integers. It works by iterating through the array, taking each element (the "key") and placing it into its correct position in the already sorted portion of the array. This involves shifting elements larger than the key to the right to make space

```
void InsertionSortTest2() {
```

```

PROFILER_ENTER("Insertion Sort Test");

constexpr int SORT_TEST_SIZE = 100000; // Size of the array to sort
std::vector<int> data(SORT_TEST_SIZE);

// Populate the array with random numbers
PROFILER_ENTER("Random Number Generation");
for (auto &num : data) {
    num = rand();
}
PROFILER_EXIT("Random Number Generation");

// Perform Insertion Sort
for (int i = 1; i < SORT_TEST_SIZE; ++i) {
    PROFILER_ENTER("Outer Loop"); // Start profiling the outer loop

    int key = data[i];
    int j = i - 1;

    // Interleaving the shifting process (Inner loop)
    while (j >= 0 && data[j] > key) {
        PROFILER_ENTER("Inner Loop (Shifting Elements)"); // Start profiling inner loop

        data[j + 1] = data[j]; // Shift element

        PROFILER_EXIT("Inner Loop (Shifting Elements)"); // End profiling inner loop
        --j;
    }

    // Place the key in its correct position
    PROFILER_ENTER("Key Assignment");
    data[j + 1] = key;
    PROFILER_EXIT("Key Assignment");

    PROFILER_EXIT("Outer Loop"); // End profiling the outer loop
}

PROFILER_EXIT("Insertion Sort Test");

// Verify sorting
bool sorted = true;
for (int i = 1; i < SORT_TEST_SIZE; ++i) {
    if (data[i - 1] > data[i]) {
        sorted = false;
    }
}

```

```

        break;
    }
}

if (sorted) {
    std::cout << "Insertion Sort successfully sorted the array." << std::endl;
} else {
    std::cout << "Insertion Sort failed to sort the array." << std::endl;
}
}

```

Profiler Structure:

Section Name: "Random Number Generation"

Purpose: Measures the time taken to populate the array with random numbers before sorting.

Section Name: "Outer Loop"

Purpose: Measures the time taken for each iteration of the outer loop that selects the key element to be inserted.

Section Name: "Inner Loop (Shifting Elements)"

Purpose: Measures the time taken to shift elements that are greater than the key to make space for insertion.

Section Name: "Key Assignment"

Purpose: Measures the time taken to place the key element into its correct position.

Section Name: "Insertion Sort Test"

Purpose: Encompasses the entire insertion sort process, including all nested profiling sections.

Profiler Stats:

```

-----
Section Name: Insertion Sort Test
Count: 1
Total Time: 1.07007
Average Time: 1.07007
Min Time: 1.07007
Max Time: 1.07007
Line Number: 292
File Name: ./Code/main.cpp
Function Name: InsertionSortTest2
-----
Section Name: Outer Loop
Count: 99

```

Total Time: 1.02862
Average Time: 0.0103901
Min Time: 0.000355
Max Time: 0.046393
Line Number: 289
File Name: ./Code/main.cpp
Function Name: InsertionSortTest2

Section Name: Inner Loop (Shifting Elements)
Count: 2186
Total Time: 0.235148
Average Time: 0.00010757
Min Time: 6.5e-05
Max Time: 0.034106
Line Number: 280
File Name: ./Code/main.cpp
Function Name: InsertionSortTest2

Section Name: Key Assignment
Count: 99
Total Time: 0.009334
Average Time: 9.42828e-05
Min Time: 6.5e-05
Max Time: 0.000163
Line Number: 287
File Name: ./Code/main.cpp
Function Name: InsertionSortTest2

Changing from Linear Search to Binary Search:

The InsertionSortTest2 function was modified to replace the linear search in the inner loop with a binary search mechanism.

This is the modified code:

```
#include <algorithm> // For std::upper_bound
```

```
void InsertionSortTest2_Modified() {  
    PROFILER_ENTER("Insertion Sort Test");  
  
    constexpr int SORT_TEST_SIZE = 100000; // Size of the array to sort  
    std::vector<int> data(SORT_TEST_SIZE);  
  
    // Populate the array with random numbers  
    PROFILER_ENTER("Random Number Generation");  
    for (auto &num : data) {
```

```

    num = rand();
}
PROFILER_EXIT("Random Number Generation");

// Perform Insertion Sort with Binary Search
for (int i = 1; i < SORT_TEST_SIZE; ++i) {
    PROFILER_ENTER("Outer Loop"); // Start profiling the outer loop

    int key = data[i];
    // Find the insertion point using binary search
    auto it = std::upper_bound(data.begin(), data.begin() + i, key);
    int j = std::distance(data.begin(), it) - 1;

    // Shift elements to make space for the key
    PROFILER_ENTER("Shifting Elements");
    while (j >= 0 && data[j] > key) {
        data[j + 1] = data[j];
        --j;
    }
    PROFILER_EXIT("Shifting Elements");

    // Place the key in its correct position
    PROFILER_ENTER("Key Assignment");
    data[j + 1] = key;
    PROFILER_EXIT("Key Assignment");

    PROFILER_EXIT("Outer Loop"); // End profiling the outer loop
}

PROFILER_EXIT("Insertion Sort Test");

// Verify sorting
bool sorted = true;
for (int i = 1; i < SORT_TEST_SIZE; ++i) {
    if (data[i - 1] > data[i]) {
        sorted = false;
        break;
    }
}

if (sorted) {
    std::cout << "Insertion Sort successfully sorted the array." << std::endl;
} else {
    std::cout << "Insertion Sort failed to sort the array." << std::endl;
}

```

```
}  
}
```

Profiler structure:

Section Name: "Random Number Generation"

Purpose: Measures the time to populate the array with random numbers before sorting.

Section Name: "Outer Loop"

Purpose: Measures the time taken for each iteration of the outer loop that selects the key element to be inserted.

Section Name: "Inner Loop (Shifting Elements)"

Purpose: Measures the time taken to shift elements that are greater than the key to make space for insertion.

Section Name: "Key Assignment"

Purpose: Measures the time taken to place the key element into its correct position.

Section Name: "Shifting Elements"

Purpose: Profiles the process of shifting elements to make space for the key

Section Name: "Insertion Sort Test"

Purpose: Encompasses the entire insertion sort process, including all nested profiling sections.

Changes Made:

The main change that was made from the insertion2 sort to the insertion2Modified sort was that the linear search in the inner loop was replaced with the binary search using `std::upper_bound` to find the appropriate insertion point. The intention behind this change was to reduce the number of comparisons needed to find the insertion point from $O(n)$ to $O(\log n)$ to potentially enhance the performance especially for larger datasets. By swiftly identifying where the key is inserted through Binary Search, we minimize the number of comparisons required to locate the insertion index

Profiler Stats

Section Name: Insertion Sort Test

Count: 1

Total Time: 1.46937

Average Time: 1.46937

Min Time: 1.46937

Max Time: 1.46937

Line Number: 500

File Name: ./Code/main.cpp

Function Name: InsertionSortTest2_Modified

Section Name: Outer Loop
Count: 99
Total Time: 1.41833
Average Time: 0.0143266
Min Time: 0.000613
Max Time: 0.058255
Line Number: 497
File Name: ./Code/main.cpp
Function Name: InsertionSortTest2_Modified

Section Name: Inner Loop (Shifting Elements)
Count: 2186
Total Time: 0.294883
Average Time: 0.000134896
Min Time: 7e-05
Max Time: 0.007339
Line Number: 488
File Name: ./Code/main.cpp
Function Name: InsertionSortTest2_Modified

Section Name: Key Assignment
Count: 99
Total Time: 0.011641
Average Time: 0.000117586
Min Time: 7.3e-05
Max Time: 0.000213
Line Number: 495
File Name: ./Code/main.cpp
Function Name: InsertionSortTest2_Modified

Analyzing Difference:

When looking at the stats, it is clear the original method is still better even though we assumed that implementing a binary search would take less time. The outer loop has an increase of 38% in its total time. The modifications essentially made in InsertionSortTest2_Modified have negatively impacted performance across all sections most notably in the outer and inner loop operations.

Changing to MoveBackward Method:

To improve the efficiency of our function, instead of shifting one element at a time, we can use the `std::move_backward()` method to move a block of all elements at once. This reduces the number of copy operations and can improve cache efficiency.

The code:

```
void InsertionSortTest2_Optimized() {
    PROFILER_ENTER("Insertion Sort Test (Optimized)");

    constexpr int SORT_TEST_SIZE = 100; // Reduced size for testing
    std::vector<int> data = {
        857, 23, 546, 19, 678, 345, 789, 12, 456, 89,
        234, 567, 90, 123, 456, 789, 234, 56, 789, 123,
        345, 678, 90, 234, 567, 890, 123, 456, 789, 12,
        345, 678, 90, 234, 567, 890, 123, 456, 789, 12,
        345, 678, 90, 234, 567, 890, 123, 456, 789, 12,
        345, 678, 90, 234, 567, 890, 123, 456, 789, 12,
        345, 678, 90, 234, 567, 890, 123, 456, 789, 12,
        345, 678, 90, 234, 567, 890, 123, 456, 789, 12,
        345, 678, 90, 234, 567, 890, 123, 456, 789, 12,
        345, 678, 90, 234, 567, 890, 123, 456, 789, 12
    };

    // Perform Insertion Sort with Binary Search and optimized shifting
    for (int i = 1; i < SORT_TEST_SIZE; ++i) {
        PROFILER_ENTER("Outer Loop (Optimized)"); // Start profiling the outer loop

        int key = data[i];

        // Find the insertion point using binary search
        auto insertion_point = std::upper_bound(data.begin(), data.begin() + i, key);

        // Shift elements to make space for the key using move_backward
        PROFILER_ENTER("Shifting Elements (Optimized)");
        std::move_backward(insertion_point, data.begin() + i, data.begin() + i + 1);
        PROFILER_EXIT("Shifting Elements (Optimized)");

        // Place the key in its correct position
        PROFILER_ENTER("Key Assignment (Optimized)");
        *insertion_point = key;
        PROFILER_EXIT("Key Assignment (Optimized)");

        PROFILER_EXIT("Outer Loop (Optimized)"); // End profiling the outer loop
    }

    PROFILER_EXIT("Insertion Sort Test (Optimized)");

    // Verify sorting
```

```

bool sorted = true;
for (int i = 1; i < SORT_TEST_SIZE; ++i) {
    if (data[i - 1] > data[i]) {
        sorted = false;
        break;
    }
}

if (sorted) {
    std::cout << "Optimized Insertion Sort successfully sorted the array." << std::endl;
} else {
    std::cout << "Optimized Insertion Sort failed to sort the array." << std::endl;
}
}

```

Changes Made:

This function improves the original implementation by leveraging two main optimizations. Instead of shifting elements one by one as in the original implementation, the optimized version uses the `std::move_backward()` function. This reduces the number of copy operations and enhances cache efficiency by moving multiple elements at once.

Profiler Stats

```

-----
Section Name: Insertion Sort Test (Optimized)
Count: 1
Total Time: 0.159009
Average Time: 0.159009
Min Time: 0.159009
Max Time: 0.159009
Line Number: 274
File Name: ./Code/main.cpp
Function Name: InsertionSortTest2_Optimized

```

```

-----
Section Name: Outer Loop (Optimized)
Count: 99
Total Time: 0.122155
Average Time: 0.00123389
Min Time: 0.001027
Max Time: 0.006152
Line Number: 271
File Name: ./Code/main.cpp
Function Name: InsertionSortTest2_Optimized

```

```

-----
Section Name: Shifting Elements (Optimized)

```

Count: 99
Total Time: 0.016226
Average Time: 0.000163899
Min Time: 0.000134
Max Time: 0.000442
Line Number: 264
File Name: ./Code/main.cpp
Function Name: InsertionSortTest2_Optimized

Section Name: Key Assignment (Optimized)
Count: 99
Total Time: 0.009321
Average Time: 9.41515e-05
Min Time: 8.7e-05
Max Time: 0.00013
Line Number: 269
File Name: ./Code/main.cpp
Function Name: InsertionSortTest2_Optimized

Analysis:

In comparison to both the InsertionSortTest2 and the InsertionSortTest2_Modified, InsertionSortTest2_Optimized has the best runtime. It took the least amount of time to sort through everything. This is because the outer loop in the optimized version is significantly faster due to the reduced number of operations in the inner loop. By finding the insertion point with a binary search (`std::upper_bound`), the algorithm cuts down the number of comparisons needed, improving efficiency for large datasets. Using `std::move_backward` instead of manually shifting elements one by one drastically reduces the time complexity and copy operations. This leads to a massive performance boost, especially in the inner loop, where the total time for shifting elements dropped from 0.235148 seconds to just 0.016226 seconds. The optimized version is 85% faster overall, making it far more efficient for large datasets.

Changing from `std::vector` to `std::list`:

The InsertionSortTest2 function was modified to replace the use of `std::vector` with `std::list` (a doubly linked list) to avoid the expensive shifting operations that occur in the inner loop. Since linked lists allow for constant-time insertions and deletions, the aim was to reduce the overall time complexity of the shifting process.

Code:

```
void InsertionSortLinkedList() {  
    PROFILER_ENTER("Insertion Sort Test (Linked List)4");  
  
    constexpr int SORT_TEST_SIZE = 100; // Reduced size for testing
```

```

std::vector<int> data = {
    857, 23, 546, 19, 678, 345, 789, 12, 456, 89,
    234, 567, 90, 123, 456, 789, 234, 56, 789, 123,
    345, 678, 90, 234, 567, 890, 123, 456, 789, 12,
    345, 678, 90, 234, 567, 890, 123, 456, 789, 12,
    345, 678, 90, 234, 567, 890, 123, 456, 789, 12,
    345, 678, 90, 234, 567, 890, 123, 456, 789, 12,
    345, 678, 90, 234, 567, 890, 123, 456, 789, 12,
    345, 678, 90, 234, 567, 890, 123, 456, 789, 12,
    345, 678, 90, 234, 567, 890, 123, 456, 789, 12,
    345, 678, 90, 234, 567, 890, 123, 456, 789, 12
};
// Perform Insertion Sort on linked list
for (auto it = std::next(data.begin()); it != data.end(); ++it) {
    PROFILER_ENTER("Outer Loop4"); // Start profiling the outer loop

    int key = *it;
    auto j = it;

    // Shift elements in the list to make space for the key
    while (j != data.begin() && *std::prev(j) > key) {
        PROFILER_ENTER("Inner Loop (Shifting Elements)4"); // Start profiling inner loop

        *j = *std::prev(j); // Shift element
        --j;

        PROFILER_EXIT("Inner Loop (Shifting Elements)4"); // End profiling inner loop
    }

    // Place the key in its correct position
    PROFILER_ENTER("Key Assignment4");
    *j = key;
    PROFILER_EXIT("Key Assignment4");

    PROFILER_EXIT("Outer Loop4"); // End profiling the outer loop
}

PROFILER_EXIT("Insertion Sort Test (Linked List)4");

// Verify sorting
bool sorted = true;
auto prev = data.begin();
for (auto it = std::next(data.begin()); it != data.end(); ++it) {
    if (*prev > *it) {

```

```

        sorted = false;
        break;
    }
    prev = it;
}

if (sorted) {
    std::cout << "Insertion Sort successfully sorted the list." << std::endl;
} else {
    std::cout << "Insertion Sort failed to sort the list." << std::endl;
}
}

```

Profiler Structure:

Section Name: "Random Number Generation" Purpose: Measures the time taken to populate the linked list with random numbers.

Section Name: "Outer Loop" Purpose: Measures the time taken for each iteration of the outer loop that selects the key element to be inserted.

Section Name: "Inner Loop (Shifting Elements)" Purpose: Measures the time taken to shift elements in the linked list during insertion.

Section Name: "Key Assignment" Purpose: Measures the time taken to place the key element in its correct position in the sorted part of the list.

Section Name: "Insertion Sort Test (Linked List)" Purpose: Encompasses the entire sorting process, including all profiling sections.

Profiler Stats:

Section Name: Insertion Sort Test (Linked List)4

Count: 1

Total Time: 0.960121

Average Time: 0.960121

Min Time: 0.960121

Max Time: 0.960121

Line Number: 358

File Name: ./Code/main.cpp

Function Name: InsertionSortLinkedList

Section Name: Outer Loop4

Count: 99

Total Time: 0.934138

Average Time: 0.00943574

Min Time: 0.000395
Max Time: 0.037899
Line Number: 355
File Name: ./Code/main.cpp
Function Name: InsertionSortLinkedList

Section Name: Inner Loop (Shifting Elements)4
Count: 2186
Total Time: 0.217962
Average Time: 9.97081e-05
Min Time: 7.7e-05
Max Time: 0.014541
Line Number: 347
File Name: ./Code/main.cpp
Function Name: InsertionSortLinkedList

Section Name: Key Assignment4
Count: 99
Total Time: 0.008057
Average Time: 8.13838e-05
Min Time: 7.1e-05
Max Time: 0.000159
Line Number: 353
File Name: ./Code/main.cpp
Function Name: InsertionSortLinkedList

Analysis:

The linked list implementation offers a modest improvement over the baseline vector version, particularly in reducing the total time and the cost of element shifting in the inner loop. However, it is still slower than the binary search-modified version and is far less efficient than the optimized version. The main drawback of the linked list is the time complexity of traversing the list in each outer loop iteration. While insertions are constant time, traversing the list is $O(n)$, which offsets the benefits of using a linked list for this sorting algorithm.